# DIGITHINGS

the digithings book

edition 26 · version 1

# CONTENTS

01 Why this book exists 3
02 Software that lives anywhere 4
03 Open source, open core 5
04 How it scales 6
05 One environment for an AI application 7
06 What you can build 8
07 digigraph, the orchestration hub 10
08 digillm, one model layer 11
09 MCP, every capability as a tool 12
10 digisearch, finding what matters 13
11 digiquant, the research vertical 14
12 digichat, the interface people touch 15
13 digikey, identity checked everywhere 16
14 digitrace, seeing what the system did 17
15 The local stack, one machine 18
16 Start on your machine 19

digithings started as one builder's own AI stack: the parts needed to use AI for real work, on a laptop or in the cloud, without a vendor deciding which model, which database or which interface that work has to use. It is published whole, under the MIT licence.

AI tooling changes every few months. A better model, a cheaper vector store or a new agent client arrives, and most applications need a rewrite to take it. This stack is shaped so they need a setting. The book explains the shape and the decisions behind it; setup and endpoints live in the documentation.

## What we hold to

**Own the stack.** Every part is code you can read, run and change. When an answer looks wrong, you can follow it to the line that made the decision.

**Location is a deployment decision.** The same services run in Docker on a laptop and in containers in a cloud account, and read the same setting names in both. Moving them changes values, not code.

**Open at the seams.** Models are reached over the OpenAI chat format, tools over MCP, identity as standard signed tokens and traces over OpenTelemetry. Each is a standard the stack did not invent, so any part can be swapped for something else that speaks it.

**Small parts, one job each.** Model access, orchestration, retrieval, identity, tracing and the interface are separate modules. A project that only needs search over its documents runs retrieval and leaves the rest off.

**Grow in place.** A solo builder and an organisation use the same contracts. Capability and cost go up as the work proves itself.

**Show the working.** Answers cite the passages they used, backtests record the engine build that produced them, and with tracing on, every request leaves a trail. Trust comes from what the system records, not from what it claims.

## How to read it

The case comes first: software that can live anywhere, an open core, a stack that scales without a rewrite, and what people build with it. Then the modules, each with what it does, where its seam is and what it costs to replace. It ends on your own machine, with the first thing to build.

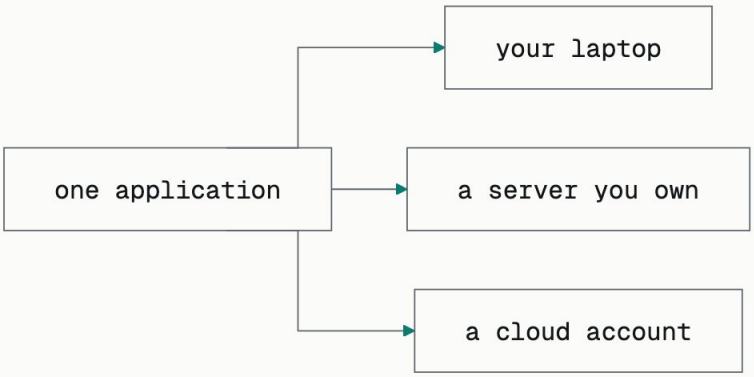
The module chapters are written for whoever will run the stack: someone at home with Docker and a settings file. If nobody on your team is, the last chapter says who does that work.

Read it through in one sitting, or open the chapter your question lives in.

An AI application should not be tied to the machine it first ran on. Over its life the model endpoint changes, the data grows and the number of people using it goes from one to many. The application should not have to change with them.

An assistant product decides where your software lives on day one: you get an account, a workspace, and your data lives in their region. An agent framework built around one vendor's runtime only runs where that runtime runs. A pile of your own scripts runs exactly where you put it and nowhere else. By the time you know what the application needs, the choice has been made for you, and undoing it is a migration project.

digithings takes the opposite position. The same code runs on a laptop, on a server you own and in a cloud account, with the same service contracts, the same setting names and the same identity model in each. The cloud home in production today is Cloudflare, where the same services run in containers behind its edge. You build locally, prove the application on real data, and move it when the work needs more capacity.




one application, three homes: the contracts between modules are the same in each

This is not an argument against the cloud. A cloud account is often the right home for a production service. The argument is against choosing infrastructure before you know what the application needs, and against infrastructure that makes the choice hard to revisit. The price is that you run the stack yourself: updates, backups and the machine are yours to keep.

Three things keep the choice open. The code is open source, so nothing is hidden behind an account. The seams between modules are standard protocols, so any module can run on a different machine from its neighbours. And no module assumes where the others live: each finds its neighbours through configuration, never through a hard-coded address.

So location stops being an architectural commitment. Stay on one machine while the work is small. Move to a cloud account when it needs more capacity. Put one module on its own hardware when it grows faster than the rest, say an index that outgrows the laptop's disk. At every step it is the code you started with.

digithings is open core. The components you would build an AI application from are published under the MIT licence: the orchestration hub, the model layer, retrieval, backtesting and optimisation, the chat interface, identity, tracing and the shared libraries. You can read them, run them, change them and ship them inside your own product.

## Why open

Open source is usually argued on price. The stronger reasons are practical.

**You can inspect it.** When an answer looks wrong, you can follow the request through code you can read, down to the line that made the decision. A closed component turns that question into a support ticket.

**You can move it.** Portability depends on knowing every assumption a component makes about where it runs. In a closed component, those assumptions are exactly what you cannot see.

**It outlives the vendor.** Software you build on keeps working whatever happens to the company that wrote it. The code is in your hands, under a licence that does not expire.

## Open at the seams

Open code is not enough if the boundaries are proprietary. A stack can publish every module and still lock you in through the way they talk to each other.

digithings keeps its seams on standards it did not write. Models are reached through the OpenAI chat format, which Ollama, LiteLLM and most providers accept. Tools are described with the Model Context Protocol, so any client that speaks it can use them. Identity travels as signed JSON Web Tokens that any service can verify with a published key, and traces leave as OpenTelemetry. Replace a module with your own that keeps its seam and the others do not notice.

## What stays closed

The closed part is narrow and has a rule. Anything a person would reasonably want to self-host for their own work stays open. What is closed is the commercial layer built on top: the finance workflows that run research cycles and portfolio decisions, the layer that turns research into live orders, the machinery for running many tenants on one deployment, and managed hosting. That layer is being separated into repositories of its own. Until the split lands, its code is still in this public one, under the same licence.

If one of those modules matters to you while it is still here, it is yours under MIT. If it has moved by the time you read this, the copy you have keeps working under that licence, because a licence does not reach back and take code away.

"Does it scale?" means one thing for a rented service and another for a stack you run. A rented service scales when someone raises a limit on a dashboard, and the bill follows. A stack you run scales when you decide what to add and where. That decision is yours, and so is the work of running what you add. The stack keeps that work small enough for one person who can run Docker.

## Two kinds of scale

The first kind is load: more people, more requests, more documents. The chat holds no conversation state of its own, so ten copies behave like one. The hub can keep its threads in Postgres but still holds rate limits and pending approvals in its own process, so today it grows on bigger hardware first.

The second kind is capability: the application learns to do more. A chat assistant gains retrieval, then a research pipeline, then tools that act on other systems. Each step is one more module switched on.

## Three dials

**Which modules run.** Start with the hub, the model layer and the chat. Add retrieval, research, a vault or tracing when the work calls for them. A module you do not run costs nothing.

**Which models answer.** A small local model is free per call and private. A large hosted model reasons better and costs per token. The model layer makes this a routing setting, so different tasks can use different models at once.

**Where it runs.** One machine, a server you own, a cloud account, or a mix. Each module can sit wherever its workload is cheapest.

| Who | What they typically run | Where the cost goes |
|---|---|---|
| A solo builder or researcher | Hub, model layer, chat and one vertical, on one machine with local models | The machine they own; hosted models only by choice |
| A team | Shared retrieval, a research pipeline and identity, on one server or a small cloud account | One server, plus models for the work that needs them |
| An organisation | Every module, tracing on all requests, several machines or a cloud account | Capacity sized per module, models routed per task |

The rows describe one system at three sizes. The code, the contracts and the setting names are the same in each, and the application built in the first row is the one running in the last.

An AI application is more than a model call. It needs a way to reach models, route work between them, find the right context, call tools, identify who is asking, show an interface and record afterwards what happened. Most projects build that scaffolding themselves before they reach the part that is theirs.

digithings is that scaffolding, built once. Each module has one job and a published contract, so you can run one of them, a few, or all of them.

| Module | Its one job |
|---|---|
| digigraph | The hub. Routes each request to the right tools and streams the answer. |
| digillm | The model layer. One way for every service to call any model. |
| digisearch | Retrieval. Indexes your documents and finds the passages that matter. |
| digivault | Documents. A folder of linked markdown notes, readable by people and tools. |
| digiquant | Research. Data, strategies, backtests and optimisation, in order. |
| digivoice | Voice. Dictation, rewriting and speech, able to run entirely on your own machine. |
| digichat | The interface. Where people ask, read answers and see artifacts. |
| digikey | Identity. Short-lived credentials every service checks itself. |
| digitrace | Tracing. A record of what every request did. |
| digibase | The shared base. Error shapes, request ids, audit redaction. |

The Model Context Protocol is the layer underneath, not a module: it is the open standard for telling an AI model which tools it may call. digigraph, digisearch, digiquant, digivault and digillm each publish their tools over it, so Claude Desktop, a code editor or an agent you wrote can use them directly. In the other direction, digigraph can call any MCP server you add. That is why the table is not a closed list.

## How the parts fit

Requests come in through the chat or an outside caller and go to the hub. The hub asks the verticals which tools they offer, runs the ones the request needs and reaches models through the model layer. digikey signs the credentials and each service checks them itself; with tracing on, every step is recorded. Each seam is a standard: the OpenAI chat format into the hub, MCP for tools, signed tokens for identity. Replace a part with your own that keeps its seam and the rest carry on. After the use cases, the chapters take the main parts one at a time.

The same modules make different applications. A use case is a choice of modules, not a separate product, and each of them starts on a single machine.

## Retrieval over your own documents

Most of what a team needs answered sits in its own files: contracts, notes, past research. Retrieval brings it to the model: your documents are split into passages, indexed by meaning, and the passages that matter are handed to the model with the question. The answer cites what it stands on, so a reader can check it.

digisearch owns the two halves, ingestion and answering, and keeps them apart. digivault holds documents as linked markdown, and digillm connects whichever model you choose. A workspace filter can travel with each query, so one index can hold several teams' documents. The same pipeline runs against a folder on a laptop and against a department's archive on managed storage.

## Financial research

A research desk needs market data, a way to test ideas against history, and results it can defend. Most tooling for this is either a spreadsheet that cannot be audited or a closed platform whose methodology you take on trust.

digiquant gives the work an ordered pipeline: data, strategy validation, backtest, optimisation and export, each stage reading only what the stage before it wrote. digigraph routes questions to it, digisearch brings in the filings and notes around a position, and digichat is where the analyst asks and reads. Every result carries its provenance, so an analyst can defend a number to someone who doubts it.

On one machine this is a private research environment for a single analyst. For a team, the pipeline moves to a shared server, backtests run on a timetable, and identity decides who can see which research.

## Voice

Speech is the input people are least willing to share, and the usual way to process it is to stream it to someone else's servers. digivoice starts from the opposite default: capture, transcription with whisper, an optional rewrite by a small local model, and speech back with Piper, all on the speaker's own Mac. The default is local: the audio is transcribed where it is spoken, and a hosted model can take its place when that is the better trade.

Because the output is ordinary text, voice becomes a way into the rest of the stack. A finished transcript pastes into whatever is in focus: a question in the chat, a research brief, made out loud while the hands are busy. digichat can also switch on the browser's own dictation and read-aloud, which is convenient but hands the audio to the browser's engine: in Chrome, Google's.

Agents and exploration

An agent is a loop that is allowed to act. It is given tools, such as search this index or open a ticket, and it picks one, reads the result and picks again until the task is done. It needs the tools, the loop, a way to ask a person before anything risky, and a record. digigraph runs the loop, MCP carries the tools, a gate per tool server decides what may run unattended, and digitrace keeps the trail.

The first agent is usually a handful of scripts a team already has, each given a name, an input schema and a description and served from a small MCP server. Claude Desktop or a code editor can connect to the same server, so the tools a team writes once work in the stack and outside it. Exploring a new idea means adding a tool, not building a new agent framework around it.

## At a glance

| Use case | Modules it runs | Starts on | Grows into |
|---|---|---|---|
| Retrieval | digisearch, digivault, digillm, digichat | A folder on a laptop | A department's archive on managed storage, with access rules per query |
| Financial research | digiquant, digigraph, digisearch, digichat | One analyst's machine | A shared research server with scheduled runs and access by role |
| Voice | digivoice, digichat | The Mac with the microphone | Any app on that machine: the transcript pastes wherever the cursor is |
| Agents | digigraph, digitrace, your MCP servers | Your own scripts as MCP tools | Many tool servers under one approval gate, shared with outside agents |

## One question, four modules

The use cases are not separate lanes. An analyst dictates a question about a holding, and digivoice turns it into text on the speaker's own machine. In the chat, the hub asks the verticals what they offer: digisearch returns the filing passages behind the position, and digiquant reruns the backtest under the stated parameters. The answer cites both, and with tracing on, the trail shows every step that produced it.

No new code was written for that question. Each module did its one job, and the application is the set of parts that were switched on. The same chain runs unchanged when the research moves to a shared server, and that is how an idea proved on one machine grows without starting again.

When a request arrives, something has to decide what happens next. A chat message, a research job and a dashboard widget all need the same loop: understand the request, pick the tools, run them in order and return the result. Without a hub, every caller writes that loop again and the copies drift apart. digigraph owns it once, so every interface shares one routing behaviour, one tool contract and one streaming format.

The hub is kept deliberately small. It routes, dispatches and streams, and leaves the real work to the verticals that own it. The price of a hub is one more hop on every request and one place where a routing mistake affects every caller.

## How it works

digigraph runs each request through a state machine built on LangGraph, with stages for research, validation, backtesting and optional optimisation. Conditional edges decide which stages a request needs: a question about documents ends after the first stage, where the tools run, and only a trading strategy goes on to the rest. Around the machine sits a registry of tools. A few live inside the hub. The verticals own the rest and publish their descriptions, the name, the inputs and what the tool does, which the hub fetches when it first needs them.




the hub asks each vertical what it offers instead of assuming it

So the description a model reads is the one the vertical ships, and a vertical can change it without a hub release. Adding a new first-party tool still means adding its handler to the hub; a tool server the operator adds over MCP needs nothing, because its tools are listed when it connects. A question about a company's filings reaches retrieval because retrieval described a document search, not because the hub was told where documents live.

## Where it fits

The hub speaks the OpenAI chat format, so digichat, Open WebUI or a short script can call it and read the streamed answer. Downstream it calls the verticals for retrieval and research, and models through the shared model layer. Identity arrives as a short-lived signed credential that the hub verifies itself rather than trusting the caller. With tracing on, every step is recorded, so a routing choice can be questioned after the fact.

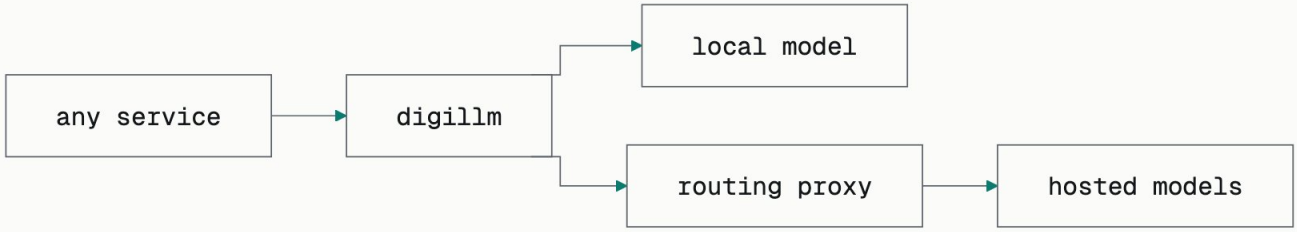
The hub holds no list of where its neighbours live, so moving a vertical to another machine changes the hub's configuration and nothing else.

Left alone, every service grows its own model client. Each picks a favourite vendor, retries in its own way and fails in its own way. A year later, switching models means touching every service, and nobody can say what a model call costs. digillm exists so that never happens: one library, used by every service, with no preference for any vendor.

With a shared layer, changing models is a configuration choice. digillm speaks the OpenAI chat format, which most providers, local runners such as Ollama and routing proxies such as LiteLLM accept, so they all look the same to the calling code. Trying a new model changes where the traffic goes, not how the code asks for it. The cost of one format is that a feature only one provider offers waits until the format can carry it.

## How it works

digillm is a library first. It ships inside the services that use it and defines the shapes every model call shares: the request, the tool call, the tool definition and structured answers checked against a schema. On top of those shapes it adds what every caller needs and nobody wants to write twice: retries, timeouts, usage accounting and optional tracing.




any service reaches any model through one layer; the provider is a setting

The library has two hard dependencies, the OpenAI client and Pydantic, and never imports a web framework. So the same code can live inside an orchestration hub, a retrieval service and a research pipeline without dragging their worlds along.

## Where it fits

The hub, retrieval and research all make their model calls through it. Because the shapes are shared, a structured answer produced in one service can be read in another without translation. When tracing is configured, usage flows into the trail; when it is not, the hooks cost nothing.

A library has no home of its own. In the local stack it points at LiteLLM, which routes each model name to Ollama on the same machine or to a hosted provider. Where models run stays an operator decision, never a code change.

An AI model on its own can only write text. To do anything else, search an index, run a backtest, open a ticket, it needs tools: named functions with described inputs that it may ask to call. The Model Context Protocol (MCP), published by Anthropic in 2024 and since adopted by many AI clients, is the open standard for offering those tools: each one has a name, a schema for its inputs and a description a model can read.

MCP is not a module, and you do not run it. It is a layer on the edge of each module that has something to offer. digigraph, digisearch, digiquant and digivault each run an MCP server beside their HTTP API, and the digillm library ships an optional one. digikey, digitrace and digibase have no tools to give, so they have no MCP server.

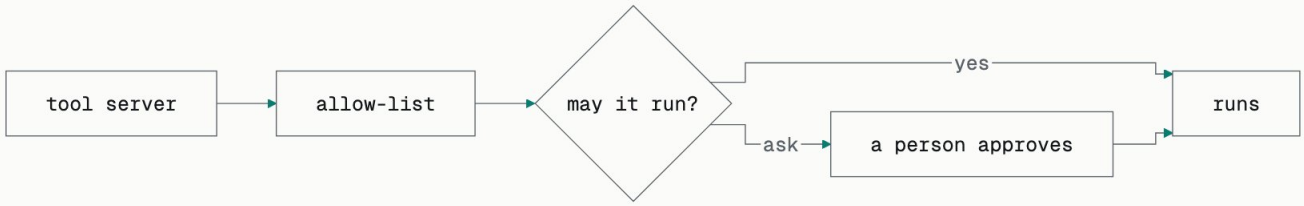
## Two directions

**Out.** Any MCP client can use those servers: Claude Desktop, a code editor, or a terminal agent you wrote, over a local pipe or across a network. No digichat and no hub sit in between. When a better client ships next year, it can search your index the day it ships.

**In.** digigraph is itself an MCP client. An operator lists tool servers, the team's tracker or someone else's product, and the hub offers their tools to the model on the next request, with no change to the hub.

## The gate

Each server the operator adds says which of its tools are allowed and which of them change something. Allowed read-only tools run unattended. A tool that changes something stops the turn and shows the exact call in the chat until a person approves or declines it.




offered and executed are checked separately: a model only sees tools it may call

The gate is checked where the tool executes, not only where it is offered, so a prompt that talks a model into a forbidden tool still fails at the gate.

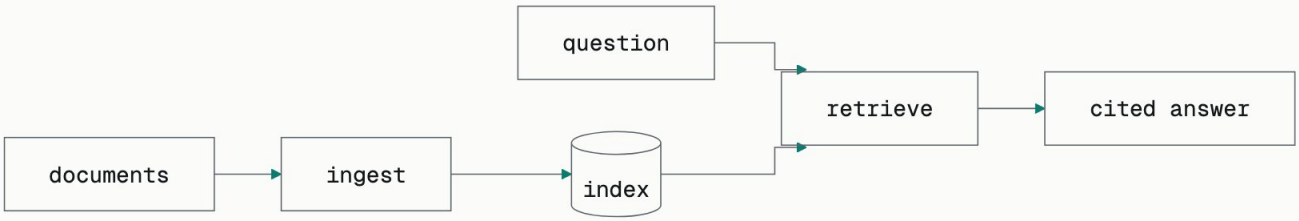
The hub's own verticals reach it over a plainer HTTP manifest instead, because the hub treats their results specially: citations, vault notes, backtest results. The trade is that a new first-party tool needs a change in the hub, while a tool on an MCP server needs none.

Language models know a great deal and remember nothing of yours. Everything an organisation knows, its documents, its history, its earlier research, sits outside the model. digisearch brings it in. It takes in your documents, indexes them by meaning and finds the right passages when a question needs them.

Without retrieval a model answers from training memory: fluent, confident and sometimes invented. With it, the answer arrives with the passages it stands on, and a sceptical reader can check them. digisearch makes that the default shape of every answer that touches private knowledge.

## How it works

Two paths stay strictly apart. Ingestion splits documents into passages, turns each into an embedding, a list of numbers that places text by meaning, and stores them in an index, so a question about staff turnover finds a passage that only says attrition. Answering takes a question, runs a keyword search and a meaning search together, reranks the results and hands the best passages to the model with the question, so the model reasons over supplied evidence instead of recalling.




two paths share an index but never logic: indexing cannot change how a question is answered

The paths share storage and nothing else, and they need different permissions: a key with `digisearch:query` can ask but cannot add documents, which takes `digisearch:ingest`. When a workspace is set, its filter is added to every query by the service, not by the caller, so one index can hold several workspaces without one reading another. The cost of the split is a step: nothing is searchable until someone ingests it.

## Where it fits

digisearch sits beside digiquant as one of the hub's research arms: quant work answers from market data, search answers from documents. The chat leans on it for grounded answers and the research pipeline calls it for background. Ingestion runs on demand, and either way it lands in the index the answering path reads.

The index backend is one setting: Chroma on local disk by default, pgvector in Postgres, Cloudflare Vectorize or Azure AI Search. In every home, ingestion is an explicit act, so nothing enters the index you did not put there. Passages do go to the model that answers, so confidential work belongs on a local one.

Quantitative research has a failure mode most software does not: results that look rigorous and are not. A backtest that peeks at the future, steps run in a convenient order, a number nobody can reproduce. Each one manufactures confidence. digiquant is built so those outcomes are hard to produce by accident, because the honest order of operations is the only order the pipeline will run.

Belief in a digiquant number rests on process. Research moves through fixed stages, each stage declares what it reads, and every result records the engine and build that produced it, so it can be checked later.

## How it works

The unit of work is the whole pipeline. A caller asks for a research task; digiquant runs the stages in order on NautilusTrader, an open trading engine with a Rust core pinned to one exact version, and returns the result with its provenance attached.




each arrow is a contract: a stage reads only what the stage before it wrote

The engine replays history one bar at a time, so a strategy cannot read a price before its moment; that is what keeps look-ahead bias out. The fixed order keeps the rest out: nothing is optimised before it has been validated and backtested. A result whose engine cannot be identified comes back marked partial and not citable, with no plausible number to mislead anyone. The price is one engine: a strategy written for another has to be ported.

## Where it fits

digiquant publishes its tasks as tools, to the hub and over MCP, so a caller can ask for research without knowing how quant work is ordered. Results go out to whoever asked, whether that is the chat, a scheduled session or a dashboard. The stack has one rule here: no other service may state a Sharpe ratio, a profit or a trade count unless it came from digiquant. Each result names the engine version, the pin and the lockfile it ran on, so a number on a dashboard leads back to the build behind it.

The pipeline runs wherever compute is cheapest for you: beside the data on one machine for fast backtests, on a timetable in the cloud, or on hardware sized to the workload. The ordering rules travel with it, because they are written into the code itself.

Most people never meet the stack. They meet digichat, the chat surface where questions go in and answers stream out. Everything behind it, the hub, the verticals and the models, serves that surface, and the surface itself does very little.

A person signs in, opens a conversation and talks. Answers stream in as they are produced, history is kept per conversation, and artifacts such as documents and visuals render beside the chat rather than inside a wall of text. The same interface can be embedded in another site, so a host can offer the experience to its own visitors with the same backend behind it.

Dictation and read-aloud are two more ways into the same conversation: the browser's own speech support, off by default, not a digithings model. Chrome sends that audio to Google, so keep dictation off for anything confidential.

## How it works

digichat is two parts that travel together: a request layer and a surface. The request layer signs the person in through any OpenID Connect provider, swaps that session for a short-lived digikey token and forwards the message to the hub with it. The surface renders the streaming answer, the history and the artifacts.

Neither half holds anything worth stealing. Sessions are verified on every request. Conversations live in the browser, and in Postgres when one is configured, never in the chat server itself. Embedded visitors are authorised per host before they see anything, and a registered host alone authorises nothing: the credential for that visit does.

## Where it fits

digichat relies on digikey for identity and on the hub for answers. The hub is its default backend, not its only one: one setting points it at an OpenAI-compatible endpoint, Anthropic, Google Vertex or Azure AI Foundry instead, which is useful for a plain chat with no tools behind it.

It works the other way too. The hub speaks the standard chat format, so any client that speaks that format and carries a valid token can sit where digichat sits.

## Where it runs

On a laptop it runs beside the stack it talks to, so development stays a single checkout. In the cloud it is a container in front of the hub and verticals; on your own servers, wherever you already serve web traffic.

In every home it stays stateless. Stop it, move it or run ten copies, and conversations survive, because the server never held them.

In a system of a dozen services, the dangerous question is who vouches for whom. If every service trusts every caller, one compromised caller owns everything. digikey is the narrow answer: exactly one service issues the signed credentials services accept, every other service checks them for itself, and no service ever takes a caller at its word.

Two kinds of caller need access. Services and outside callers hold long-lived access keys (they start `dgk_live_`), shown once when issued, and exchange them for a signed token that lasts 15 minutes. People sign in through the interface, and the request layer trades their session for the same short-lived credential. Both come from the same issuer.

## How it works

Issuing is central and checking is local. The identity service looks at what it is shown, an operator credential, a session or a machine credential, and returns a short-lived signed credential carrying identity, tenant and scopes. Every other service fetches the issuer's public signing keys, keeps them for five minutes, and checks each incoming token itself: issuer, audience, expiry and scopes. No call home is needed per request, so if the issuer goes quiet for a few minutes, requests keep working.




one service signs; every other service verifies locally with the public keys

Access keys are stored only as bcrypt hashes, so a lost one is replaced, never recovered. Scopes keep every credential narrow: a key with `digisearch:query` can search documents but cannot add them, because it was never granted `digisearch:ingest`.

## Where it fits

Everything protected meets digikey: the hub, the verticals and the data paths. Each service declares which scopes its routes need, and shared code enforces them the same way everywhere. Outside callers come in through the same exchange, so there is no side door with different rules.

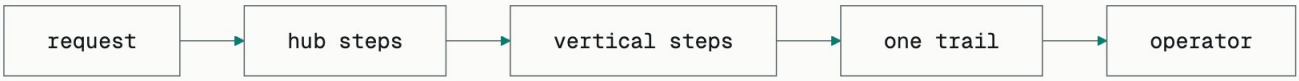
Identity is a service of its own, rather than code in each service, so there is one place to issue, list and revoke. That makes it a single point of trust, and the design contains it. Tokens expire in minutes, scopes are narrow, and revoking a key also blocks the tokens already issued from it. The price is another service to run, and rotating the signing key today means a redeploy. Because checking is local, identity works the same on one machine, on your own servers and in a cloud account.

When an answer looks wrong, someone has to be able to ask which tools ran, in what order, what each one saw and how long it took. digitrace is how digithings answers: it records the trail of every request as it happens, so nobody has to piece it together from log lines afterwards.

The trail serves several readers. An operator uses it to debug a bad answer. A reviewer uses it to audit a research number. Anyone deciding whether to trust the system can see the steps behind a result. And the trail shows where to improve: a tool that takes four seconds, or a call that fails one time in ten, stands out at once.

## How it works

digitrace has two faces. As a library, it gives every service a thin helper that emits OpenTelemetry data to whatever endpoint the deployment names. OpenTelemetry is the open standard for traces, so services depend on no tracing vendor. As a small service, it reports the tracing state to the hub and the dashboards, so an operator can see it without touching sensitive configuration.




one request, one story: the trail follows it across every service it touches

The trail follows a request across service boundaries: each call carries the standard `traceparent` header, so a step in the hub and the step it triggered in a vertical appear in one story rather than in two logs that have to be matched by timestamp.

## Where it fits

Every service that acts carries the helper, so where a trace goes is one setting, not a code change. Because the format is OpenTelemetry, switching to a different tracing backend changes configuration, not service code.

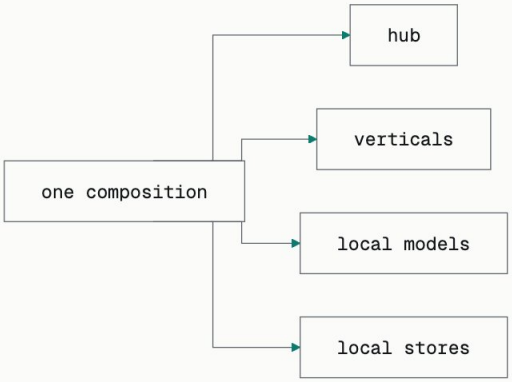
The trail goes wherever you point it: a backend on your own machine, one beside your cloud workers, or one on your own servers with the retention you choose. Tracing every request costs some storage and a little time per request, far less than debugging a multi-service answer from scattered logs.

Before the system runs anywhere else, it runs here: one machine, every service beside its neighbours, the full loop from question to answer on hardware you can touch. The local stack is not a demo mode or a trial with features removed. It is the same system arranged for one machine.

You need Docker and about 20 GB of free disk; a local model also wants plenty of memory. Services start side by side, talk over the machine's own network and keep their data in local stores. A developer runs the same composition they ship, and a small firm can run the whole product on one well-chosen computer. There is no digithings account. Model calls go wherever you point them: a provider with your own key, or Ollama on the machine, one setting apart. Which one you choose decides where your text goes.

## How it works

One Docker Compose file describes which services run, what each needs and how they reach each other. `make up` starts the core: identity, the hub, research, retrieval, tracing status, the model router and Ollama. The chat, the vault, the MCP servers and a trace collector are profiles you add. Stopping it puts everything away. Sample documents ship with it and load with one command.




one file describes the whole machine; the services inside are the ones that run everywhere

The services behave exactly as they do in any other home, and a request is signed and checked exactly as in a cloud account, because the code never learns where it is running. The only local concessions are friendlier defaults and warm caches.

## Where it fits

The local stack is where the working day happens: development, experiments, demonstrations and small production workloads. digithings runs its own production on the cloud home instead; nothing in the stack insists that a client do the same. The last chapter covers what happens when one machine is no longer enough. Nothing written against the local stack has to be rewritten, so work done locally carries straight over.

The laptop is usually treated as a developer convenience, a smaller and vaguer copy of the real system. That is backwards. If an application cannot run on your own computer, it is hard to trust, hard to debug and hard to hand to anyone else. So the first deployment of anything built on digithings is the computer in front of you.

## The first hour

Choose the model first: a hosted provider's key, or the one setting that keeps every call on the machine with Ollama. That choice decides where your passages go, so make it before you load anything of your own. Then start the local stack with the chat and trace profiles, load the sample documents with one command and ask the chat a question about them. The first thing you see is the whole loop working: the interface, the hub, retrieval, a model and a trace of what happened.

## The first week

Pick the one use case closest to your work and switch on only what it needs. For research, that is the research vertical and retrieval over your own notes. For documents, retrieval and the vault. For an agent, the tools you already have, described over MCP so the hub can call them. Keep the trace on and read it. It shows how the system reached each answer, which is the fastest way to learn where to change it.

## When it outgrows the machine

You will know, because the trace and the machine both tell you: a pipeline that needs more memory, an index that has grown past the disk, more people than one machine should serve. At that point, move the module that is under pressure, not the whole stack. Put it on a server you own or in a cloud account, point its neighbours at the new address (one setting per neighbour) and carry on. Its tools, its settings and its credentials work there unchanged.

## Where it goes

The design decisions behind each module, with references into the code, are at digithings.ai/book. The code is open under the MIT licence, and every module carries its own architecture notes.

That is also the answer to who keeps it running. Nothing here depends on one person staying interested: the code and the documentation are public, the work of improving it is legible to an agent as well as to a person, and the stack carries the skills, tools and review loops it is maintained with. The next maintainer can be a person or an agent, and neither starts from a blank page.